

CZOI Toolkit — User Guide

Version 1.0 · A guide to building secure, intelligent organizational systems with the Constrained Zoned-Object Architecture

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Your First CZOI System](#)
 4. [Core Concepts](#)
 5. [Building Zone Hierarchies](#)
 6. [Roles, Users, and Operations](#)
 7. [The Permission Calculus](#)
 8. [Constraints with UniLog](#)
 9. [Daemons and Signals](#)
 10. [Neural Components](#)
 11. [Semantic Embeddings](#)
 12. [Adaptive Access Control](#)
 13. [Audit and Observability](#)
 14. [Persistence and Web Frameworks](#)
 15. [Complete Worked Example](#)
 16. [Cookbook](#)
 17. [Troubleshooting](#)
 18. [Best Practices](#)
 19. [API Reference](#)
-

1. Introduction

What is CZOI?

The **CZOI toolkit** is the reference Python implementation of the **Constrained Zoned-Object Architecture (CZOA)** — a unified framework for building systems that are simultaneously:

- **Secure** — enforce strict, auditable access policies.
- **Intelligent** — adapt to changing conditions through neural components.
- **Organisation-aligned** — mirror the hierarchical structure of real enterprises.

The core idea is simple but powerful: **every organizational unit is a zone, and every zone is itself a full system**. A hospital is a zone. So is its Emergency department. So is a specific bed in the Emergency department. Each has its own roles, operations, constraints, daemons, and neural components. Each inherits from its parent. Each can be developed, tested, and deployed independently.

Who is this guide for?

Software engineers, system architects, and researchers who need to build:

- Enterprise access-control systems with formal security guarantees.
- Adaptive systems that respond to changing conditions without violating policy.
- System-of-systems architectures where each division retains local autonomy under global governance.

What you need to know

- **Python:** comfort with classes, decorators, and async ([asyncio](#)).
- **Basics of RBAC:** what roles and permissions are.
- No prior knowledge of CZOA, UniLog, or category theory is required.

Reading this guide

Sections 3–7 are the **core tutorial** — read them in order. Sections 8–13 are **feature guides** you can dip into as needed. Sections 15–19 are **reference** material.

Every code snippet in this guide is self-contained and runnable. You can copy them into a Python file and execute them.

2. Installation

Requirements

- Python 3.9 or later
- The [unilog-toolkit](#) package (installed automatically)

Standard install

```
pip install czoι-toolkit
```

With optional features

```
pip install czoι-toolkit[neural]      # scikit-learn for RoleMiner
pip install czoι-toolkit[embedding]  # sentence-transformers for semantic
embeddings
pip install czoι-toolkit[dev]        # pytest, ruff, mypy
```

Verify the installation

```
import czoι
print(czoι.__version__)  # 1.0.0
```

If this prints without error, you're ready. If you see `UniLogBridgeError` later, check that `unilog-toolkit` is installed:

```
pip show unilog-toolkit
```

From source

```
git clone https://github.com/hongxueharriswang/czoi-toolkit.git
cd czoi-toolkit
pip install -e ".[dev,neural,embedding]"
```

3. Your First CZOI System

Let's build the simplest possible CZOI system: a company with an HR department, two roles, and a permission check.

```
from czoi import (
    Application, CZOABuilder, Decision, Operation, Role, User,
)

# -----
# 1. Create the system and its zones
# -----
builder = CZOABuilder("AcmeCorp")
company = builder.root
hr      = builder.add_zone("HR", parent=company, atomic=True)

# -----
# 2. Define an application and its operations
# -----
hr_app = Application("HRSysTem", zone=hr)
view_employee = hr_app.add_operation(Operation("view_employee"))
edit_employee = hr_app.add_operation(Operation("edit_employee"))
hr.add_application(hr_app)

# -----
# 3. Create roles
# -----
manager = Role("Manager", zone=hr,
               base_permissions=[view_employee, edit_employee])
assistant = Role("Assistant", zone=hr,
                 base_permissions=[view_employee])
manager.add_junior(assistant) # Manager is senior to Assistant

hr.add_role(manager)
hr.add_role(assistant)
```

```

# -----
# 4. Create users (containment principle enforced automatically)
# -----
alice = User("alice", roles={"Assistant"})
bob   = User("bob",   roles={"Manager"})

company.add_user(alice)      # register at root first
hr.add_user(alice)           # then at the child zone

company.add_user(bob)
hr.add_user(bob)

# -----
# 5. Check permissions
# -----
engine = builder.permission_engine

print(engine.decide(alice, view_employee, hr).name) # ALLOW
print(engine.decide(alice, edit_employee, hr).name) # INCONCLUSIVE
print(engine.decide(bob,   edit_employee, hr).name) # ALLOW

```

What just happened?

1. **CZOABuilder** creates the root zone and wires together a permission engine, constraint manager, daemon manager, and embedding service. Every zone you add inherits these automatically.
2. **Role.add_junior()** establishes the intra-zone seniority relation. Because Manager is senior to Assistant, Bob inherits all of Alice's permissions on top of his own.
3. **zone.add_user()** enforces the containment principle: a user must be registered at the parent before any child can hold them. This is what makes the hierarchy coherent.
4. **engine.decide()** returns a three-valued **Decision**:
 - **ALLOW** — the user has a covering role in this zone.
 - **DENY** — the user has a covering role but a constraint forbids it.
 - **INCONCLUSIVE** — no covering role found locally; a parent might override.

4. Core Concepts

The 10-tuple

Every CZOI zone is a 10-tuple:

$$S = (Z, R, U, A, O, N, E, \Gamma, \Phi, \Delta)$$

Symbol	Name	What it holds
Z	Zones	Child subsystems (recursive)
R	Roles	Job functions with base permissions

Symbol	Name	What it holds
U	Users	Identities with roles and attributes
A	Applications	Structural modules (deployable units)
O	Operations	Atomic executable actions
N	Neural	Trainable functions (predictors, detectors)
E	Embeddings	Semantic vectors + alignment
Γ	Constraints	Identity, trigger, goal, access rules
Φ	Permissions	Two-stage recursive decision function
Δ	Daemons	Continuous monitoring processes

These ten components are **orthogonal**: each has a distinct role and none of them overlaps with another. This is what makes CZOI minimal — the framework doesn't ask you to learn a dozen extra concepts that could be folded into these ten.

AtomicZone vs CompositeZone

Every zone is either:

- **CompositeZone** (the default) — may contain child zones. `Z_z` may be non-empty.
- **AtomicZone** — a leaf. Calling `add_zone` on it raises `TypeError`. `Z_z = ∅`.

```
# Explicit composite
hospital = builder.add_zone("Hospital")           # CompositeZone

# Explicit atomic
emergency = builder.add_zone("Emergency", parent=hospital, atomic=True)

# This would raise TypeError:
# emergency.add_zone(AtomicZone("something"))
```

Applications vs Operations

This is the most important design decision in CZOI:

- An **application** is a *structural module*. It groups related operations and is the natural unit of deployment.
- An **operation** is an *atomic permission target*. It is the only thing you can grant a role.

```
app = Application("Payroll", zone=hr)
process = app.add_operation(Operation("process_payroll"))
adjust = app.add_operation(Operation("adjust_salary"))
hr.add_application(app)
```

```
# You grant permissions on OPERATIONS, not on applications:
admin_role.grant(process)      # correct
admin_role.grant(adjust)      # correct
# admin_role.grant(app)        # WRONG – will raise or misbehave
```

Why this matters:

- **Minimality:** if applications were permission targets, you'd need two parallel permission systems — one for apps and one for ops. CZOI avoids that.
- **Granularity:** you can grant a role the ability to `view_payroll` without granting `edit_payroll`, even if they're in the same application.
- **Auditing:** every access log entry names a specific operation, not a vague "used the Payroll app."

5. Building Zone Hierarchies

Creating the tree

```
builder = CZOABuilder("University")

uni      = builder.root
coe      = builder.add_zone("CollegeOfEngineering", parent=uni)
cs       = builder.add_zone("CSDepartment", parent=coe, atomic=True)
registrar = builder.add_zone("Registrar", parent=uni, atomic=True)
```

Result:

```
University (composite)
├── CollegeOfEngineering (composite)
│   └── CSDepartment (atomic)
└── Registrar (atomic)
```

Walking the tree

```
for zone in builder.root.walk():
    print(f"{' ' * zone.depth()}- {zone.name}")

# University
# - CollegeOfEngineering
#   - CSDepartment
#   - Registrar
```

The containment principle

Every child zone's user set is a subset of its parent's:

```
U_child ⊆ U_parent
```

This is enforced automatically. When you register a user:

```
user = User("alice", roles={"Student"})

uni.add_user(alice)      # OK – root
coe.add_user(alice)      # OK – parent already has alice
cs.add_user(alice)       # OK – parent already has alice

# Attempting to register at a child before the parent raises:
# cs.add_user(other_user) # raises ZoneContainmentError
```

Ancestry and depth

```
cs.ancestry()           # [University, CollegeOfEngineering, CSDepartment]
cs.depth()               # 2
cs.is_ancestor_of(cs)   # True
uni.is_ancestor_of(cs)  # True
```

Properties on zones

Every zone has a typed property store:

```
cs.properties.set("capacity", 500, type_hint="int")
cs.properties.set("building", "Turing Hall", type_hint="str")

cs.properties.get("capacity")           # 500
cs.properties.get("building")           # "Turing Hall"
cs.properties.get("missing", default=0) # 0
```

The type hint enforces correctness:

```
cs.properties.set("capacity", "many", type_hint="int")
# TypeError: capacity: expected int, got <class 'str'>
```

Properties are visible to constraints and daemons — they are the "state" of your system.

6. Roles, Users, and Operations

Creating roles

```
doctor = Role(  
    "Doctor",  
    zone=hospital,  
    base_permissions=[prescribe, view_patient, order_lab],  
)  
hospital.add_role(doctor)
```

A role always belongs to a specific zone. It is inherited by all descendant zones automatically:

```
# If Hospital has child zone Emergency:  
emergency.roles["Doctor"] is doctor    # True – same role object
```

Seniority (intra-zone inheritance)

```
attending = Role("Attending", zone=hospital, base_permissions=[prescribe])  
resident = Role("Resident", zone=hospital, base_permissions=  
    [view_patient])  
  
attending.add_junior(resident)  
# Attending now inherits everything Resident can do, plus prescribe.
```

You can chain seniority:

```
chief.add_junior(attending)    # Chief inherits Attending's permissions  
# Chief now transitively inherits Resident's permissions too.
```

Granting and revoking permissions

```
# Direct (fast, no cache invalidation)  
role.grant(operation)  
role.revoke(operation)  
  
# Through the zone (invalidates the permission engine's cache)  
hospital.grant(role, operation)  
hospital.revoke(role, operation)
```

Prefer the zone-level methods whenever you're in a running system — they keep the permission cache consistent.

Creating users

```
alice = User(
    "alice",
    roles={"Doctor"},
    attributes={
        "employee_id": "E-1234",
        "shift": "day",
        "specialty": "cardiology",
    },
    credentials={"password_hash": "..."},
)
```

Users carry:

- **roles** — a set of role *names*. Zones resolve them locally.
- **attributes** — a typed property store, like zone properties.
- **credentials** — anything you want (password hash, tokens, etc.).

```
alice.roles                # {"Doctor"}
alice.has_role("Doctor")   # True
alice.attributes.get("specialty") # "cardiology"
alice.attributes.set("on_call", True)
```

Registering users

```
# Always register along the containment path
for zone in [root, hospital, emergency]:
    zone.add_user(alice)
```

Or use a helper:

```
def register_along_path(user, zone):
    for z in zone.ancestry():
        if user.name not in z.users:
            z.add_user(user)

register_along_path(alice, emergency) # adds to Emergency, Hospital, Root
```

7. The Permission Calculus

Basic decisions

```
from czoι import Decision

decision = engine.decide(user, operation, zone)
# Decision.ALLOW / Decision.DENY / Decision.INCONCLUSIVE
```

Three values, not two:

- **ALLOW** — the user has a covering role here.
- **DENY** — the user has a covering role, but a constraint forbids the operation.
- **INCONCLUSIVE** — no covering role in this zone. A parent might still grant.

The recursive structure

When you call `decide(user, op, zone)`:

1. The engine checks the user's roles **in this zone**.
2. If a role covers the operation, it evaluates the zone's access constraints.
3. If constraints pass → **ALLOW**.
4. If a role covers but constraints fail → **DENY**.
5. If no role covers → the engine recurses to the parent zone.
6. If the root is reached and still inconclusive → **DENY**.

```
# Suppose only Hospital has the Doctor role, and the user is registered
# in Emergency (a child of Hospital). The engine will:
#   1. Look in Emergency → no Doctor role → INCONCLUSIVE
#   2. Recurse to Hospital → Doctor role found → ALLOW
print(engine.decide(alice, prescribe, emergency).name)  # ALLOW
```

Effective permissions

The engine computes the user's effective permission set as:

```
P_effective(role, zone) =
  P_base(role)
  ∪ ⋃_{r' ∈ junior(role)} P_base(r')
  ∪ ⋃_{γ : (zone, role) → (zone', role')} P_base(role')
```

In code, this is:

```
def effective(role):
    ops = set(role.base_permissions)
    for junior in role.junior_roles:
        ops |= effective(junior)  # recursive
    return ops
```

You can inspect it directly:

```
perms = {op.qualified_name for op in effective(attending)}
print(perms)
# {'EMR.prescribe', 'EMR.view_patient', 'EMR.order_lab'}
```

Caching

The engine caches decisions by `(user, operation, zone)`. This gives sub-millisecond lookups in production:

```
print(engine.stats())
# {'hits': 43, 'misses': 1180, 'parent_lookups': 4, 'denies': 0}
```

The cache is invalidated automatically when you:

- Call `zone.grant(role, op)` or `zone.revoke(role, op)`.
- Add a new role to a zone.
- Add a new user to a zone.
- Register a new operation.

If you mutate a role's `base_permissions` directly (via `role.grant(op)`), the cache is **not** invalidated. Prefer the zone-level methods.

Enabling audit

```
from czoι import PermissionEngine

builder.permission_engine = PermissionEngine(
    cache_enabled=True,
    audit_enabled=True,
)
for zone in builder.root.walk():
    zone.set_permission_engine(builder.permission_engine)

# ... run some decisions ...

for record in builder.permission_engine.audit[-5:]:
    print(f"{record.timestamp} {record.user} "
          f"{record.operation} @ {record.zone} → {record.decision.name}")
```

Each `DecisionRecord` has:

Field	Meaning
<code>timestamp</code>	UTC datetime

Field	Meaning
<code>user</code>	User name
<code>operation</code>	Qualified operation name
<code>zone</code>	Zone name
<code>decision</code>	<code>Decision</code> enum
<code>stage</code>	"local", "parent-override", "root-deny", or "cache"

8. Constraints with UniLog

CZOA defines four families of constraints — $\Gamma = (\mathbf{I}, \mathbf{T}, \mathbf{G}, \mathbf{C})$:

- **I** — Identity: invariants that must always hold.
- **T** — Trigger: event-condition-action rules.
- **G** — Goal: optimisation objectives.
- **C** — Access: permission-related rules.

The toolkit delegates constraint specification to the **UniLog toolkit**, a fibred logic framework with eleven solvers.

Adding a constraint

```
builder.add_access_constraint("""
    signature {
        sort User, Role;
        constant Doctor : Role;
        constant Auditor : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, Doctor) and hasRole(u, Auditor))
""")
```

The syntax is UniLang — first-order logic with temporal, modal, and deontic operators. The `signature` block declares the sorts, constants, functions, and predicates you'll use. Everything after is the constraint itself.

The four kinds

```
# Identity – an invariant
builder.add_identity_constraint("""
    signature { sort Zone; predicate valid(z: Zone); }
    forall z: Zone . valid(z) -> valid(z)
""")
```

```

# Trigger – event-condition-action
builder.add_trigger_constraint("""
    signature {
        sort Patient;
        predicate critical(p: Patient);
        predicate alerted(p: Patient);
    }
    forall p: Patient . critical(p) -> alerted(p)
""")

# Goal – optimisation objective
builder.add_goal_constraint("""
    signature {
        sort KPI;
        predicate minimised(k: KPI);
        predicate optimised(k: KPI);
    }
    forall k: KPI . minimised(k) -> optimised(k)
""")

# Access – permission policy
builder.add_access_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
""")

```

Built-in predicates

The `CZOIModel` automatically exposes:

Predicate	Meaning
<code>inZone(u, z)</code>	User <code>u</code> is registered in zone <code>z</code>
<code>childOf(c, p)</code>	Zone <code>c</code> is a direct child of zone <code>p</code>
<code>hasRole(u, r)</code>	User <code>u</code> holds role <code>r</code>
<code>canPerform(u, o)</code>	User <code>u</code> is allowed to perform operation <code>o</code>

And one built-in function:

Function	Meaning
<code>parent(z)</code>	The parent zone of <code>z</code> (or <code>z</code> itself for the root)

Custom predicates

You can register your own predicates to bridge business rules into the constraint language:

```

builder.register_predicate(
    "onShift",
    lambda u: u.attributes.get("shift") == "day",
)

builder.register_predicate(
    "zoneFull",
    lambda z: len(z.users) >= z.properties.get("capacity", 999),
)

builder.add_access_constraint("""
    signature {
        sort User, Zone;
        predicate onShift(u: User);
        predicate zoneFull(z: Zone);
        predicate inZone(u: User, z: Zone);
    }
    forall u: User, z: Zone .
        (inZone(u, z) and zoneFull(z) and not onShift(u))
        -> not inZone(u, z)
""")

```

The `CZOIModel` evaluates predicates against the live system — the constraint sees the real state, not a snapshot.

Checking constraints manually

```

results = builder.constraint_manager.check_all()
for kind, ok in results.items():
    print(f"{kind}: {'OK' if ok else 'VIOLATED'}")

```

Or against a specific zone:

```

results = builder.constraint_manager.check_zone(emergency)

```

Constraint evaluation hooks

Access constraints (`C` kind) are evaluated automatically inside the permission engine. When you call `engine.decide(...)`, the engine:

1. Finds a covering role.
2. Calls `constraint_manager.is_satisfied(user, op, zone)`.
3. If any access constraint fails → **DENY**.

This is why you get **DENY** vs **INCONCLUSIVE**: the former means a constraint blocked you; the latter means no role covered you.

9. Daemons and Signals

Daemons (Δ) are continuous monitors. They run periodically, read the system state, and emit typed signals up a daemon tree.

The Daemon base class

```
from czoι import Daemon, DaemonSignal

class BatteryDaemon(Daemon):
    def __init__(self, robots, threshold=0.2, parent=None):
        super().__init__("BatteryDaemon", parent=parent, interval=1.0)
        self.robots = robots
        self.threshold = threshold

    def monitor(self):
        for r in self.robots:
            batt = r.attributes.get("battery", 1.0)
            if batt < self.threshold:
                self.emit_signal(
                    DaemonSignal.STATE_WARNING,
                    {"robot": r.name, "battery": batt},
                )
```

The `monitor()` method is called periodically by the `DaemonManager`. It should be synchronous (no `async`).

The signal hierarchy

```
class FleetDaemon(Daemon):
    def __init__(self, parent=None):
        super().__init__("FleetDaemon", parent=parent, interval=2.0)
        self.warnings = []
        self.criticals = []

    def on_signal(self, signal, payload, source=None):
        if signal is DaemonSignal.STATE_WARNING:
            self.warnings.append((source.name, payload))
        elif signal is DaemonSignal.STATE_CRITICAL:
            self.criticals.append((source.name, payload))
```

The default `handle_signal` calls `on_signal` locally, then propagates to `self.parent`. This gives you a natural tree of escalation:

```
FleetDaemon (root)
├─ BatteryDaemon    – emits STATE_WARNING on low battery
```

```
└─ OvercurrentDaemon – emits STATE_CRITICAL on overcurrent
└─ ReportDaemon      – emits informational signals
```

Attaching daemons

```
fleet = FleetDaemon()
battery = BatteryDaemon(robots, parent=fleet)
overcurrent = OvercurrentDaemon(zones, parent=fleet)

for d in (fleet, battery, overcurrent):
    builder.add_daemon(d)
```

`builder.add_daemon()` registers the daemon with the root zone **and** with the `DaemonManager` — you don't need to do both.

Running daemons

Synchronous — for tests and step-based simulations:

```
for step in range(60):
    sim.step()
    builder.daemon_manager.tick()    # invokes every monitor once
```

Asynchronous — for production:

```
import asyncio

async def main():
    await builder.daemon_manager.run(duration=300.0)

asyncio.run(main())
```

The `DaemonManager` runs each daemon's `monitor()` in a thread from a `ThreadPoolExecutor`, so a slow monitor never blocks the event loop.

Per-daemon intervals

Each daemon declares its own interval (in seconds):

```
BatteryDaemon(robots, parent=fleet, interval=0.5)    # every 500 ms
OvercurrentDaemon(zones, parent=fleet, interval=2.0) # every 2 s
```

The `DaemonManager` schedules each daemon independently.

Error isolation

`Daemon.safe_monitor()` wraps `monitor()` in a try/except. A daemon that raises does not crash its siblings:

```
def safe_monitor(self):
    if not self.enabled:
        return
    try:
        self.monitor()
    except Exception:
        log.exception("Daemon %s raised", self.name)
```

The `DaemonManager` always calls `safe_monitor`, never `monitor` directly.

10. Neural Components

The toolkit ships with three neural primitives plus a wrapper for arbitrary models.

Predictor

A trainable linear model with sigmoid output. Good for binary classification and probability estimation.

```
from czoil import Predictor
import numpy as np

# Option 1: fit from data
predictor = Predictor("sepsis", threshold=0.85)
X = np.array([...])          # shape (n, features)
y = np.array([...])          # shape (n,), values in {0, 1}
predictor.fit(X, y, epochs=500, lr=0.05)

predictor.predict({"hr": 0.8, "temp": 0.9, "lactate": 0.7})
# -> float in [0, 1]

predictor.fires({"hr": 0.8, "temp": 0.9, "lactate": 0.7})
# -> bool (predict >= threshold)

# Option 2: wrap an existing function
predictor = Predictor("rule", fn=lambda f: 1.0 if f["x"] > 5 else 0.0)
```

You can also pass numpy arrays directly:

```
predictor.predict(np.array([0.8, 0.9, 0.7]))
```

AnomalyDetector

An autoencoder trained on normal data. Flags samples whose reconstruction error exceeds a calibrated threshold.

```
from czo import AnomalyDetector
import numpy as np

# 1. Collect normal samples
X_normal = np.random.normal(0, 1, size=(1000, 5))

# 2. Build and fit the detector
detector = AnomalyDetector(
    name="access_anomaly",
    input_dim=5,
    latent_dim=3,
    threshold=0.5,      # will be recalibrated in fit()
    seed=42,
)
detector.fit(X_normal, epochs=300, lr=0.05)

# 3. Score new samples
sample = np.array([10.0, 10.0, 10.0, 10.0, 10.0])
detector.score(sample)      # float (reconstruction error)
detector.is_anomalous(sample) # bool (score > calibrated threshold)
```

After `fit`, `detector.threshold` is set to the 95th percentile of training scores — the toolkit's equivalent of `contamination=0.05`.

Important: always z-score normalise your features before fitting. The autoencoder uses tanh activations; unscaled features dominate the loss.

```
mean = X_normal.mean(axis=0)
std = X_normal.std(axis=0)
X_normal_norm = (X_normal - mean) / std
```

RoleMiner

Unsupervised discovery of role structures from historical access logs. Implements the paper's §5.1 pipeline: autoencoder + clustering.

```
from czo import RoleMiner
import numpy as np

# Binary matrix: rows = users, columns = operations
# X[i, j] = 1 iff user i has ever used operation j
X = np.array([
    [1, 1, 0, 0], # user 0
    [1, 1, 0, 0], # user 1
    [0, 0, 1, 1], # user 2
```

```

    [0, 0, 1, 1],    # user 3
], dtype=float)

op_names = ["A.read", "A.write", "B.read", "B.write"]

miner = RoleMiner(latent_dim=2, min_cluster_size=2, seed=42)
result = miner.mine(X, op_names, min_support=0.5)

print(result.suggested_roles)
# {'MinedRole_0': ['A.read', 'A.write'],
#  'MinedRole_1': ['B.read', 'B.write']}
print(result.confidence)
# {'MinedRole_0': 1.0, 'MinedRole_1': 1.0}
print(result.n_clusters)    # 2

```

The miner uses `sklearn.cluster.HDBSCAN` when available, falling back to `AgglomerativeClustering`.

Custom neural components

Any object with the right interface works as a neural component:

```

class MyModel:
    def predict(self, features) -> float:
        ...

zone.add_neural("my_model", MyModel())

```

You can then reference it in daemons, constraint predicates, or a `NeuralContribution`:

```

def hook(user, operation, zone, base):
    model = zone.neural.get("my_model")
    if model and model.predict({"user": user.name}) > 0.9:
        return Decision.DENY
    return base

```

11. Semantic Embeddings

The `EmbeddingService` maps entities (operations, roles, zones) to vectors in a Hilbert space so that semantic similarity becomes geometric proximity.

Basic usage

```

from czoι import EmbeddingService

emb = EmbeddingService(dimension=64)    # hash-based by default

```

```
v_op    = emb.embed_operation(operation)
v_role  = emb.embed_role(role)

similarity = emb.similarity(v_op, v_role)
```

With transformer backend

For semantically meaningful embeddings, install `sentence-transformers` and request the transformer:

```
emb = EmbeddingService(
    model_name="all-MiniLM-L6-v2",
    use_transformer=True,
)
```

Now embeddings capture real semantic content:

```
v1 = emb.embed("patient_discharge_summary")
v2 = emb.embed("clinical_discharge_note")
v3 = emb.embed("payroll_processing")

emb.similarity(v1, v2)    # high – both are discharge documents
emb.similarity(v1, v3)    # low – unrelated domains
```

Global alignment functor

The paper's `E_align` projects local embeddings into a shared space, enabling cross-zone similarity. In the toolkit:

```
v_local = emb.embed("Emergency:attending_physician")
v_global = emb.align_to_global(v_local)
```

By default this is a unit-normalisation. You can train an alignment matrix on labelled positives and negatives:

```
positives = [
    (emb.embed("attending_physician"),
     emb.embed("doctor")),
    (emb.embed("nurse"),
     emb.embed("registered_nurse")),
]
negatives = [
    (emb.embed("attending_physician"),
     emb.embed("payroll_clerk")),
]
```

```
emb.train_alignment(positives, negatives,
                    epochs=200, lr=0.01, margin=0.5)

# Persist the alignment layer
emb.save_alignment("alignment.npy")
emb.load_alignment("alignment.npy")
```

Use case: cross-zone role matching

Give a senior role in one zone the permissions of a semantically similar role in another zone:

```
def match_roles(source_role, target_zone, threshold=0.75):
    source_vec = emb.embed_role(source_role)
    for role in target_zone.roles.values():
        target_vec = emb.embed_role(role)
        if emb.similarity(source_vec, target_vec) > threshold:
            yield role

# Apply a gamma-like mapping based on semantics
for matched in match_roles(attending, icu_zone):
    for op in matched.base_permissions:
        attending.grant(op)
```

This is the paper's §5.2 "cross-zone understanding" pattern.

12. Adaptive Access Control

The toolkit's signature capability: **permissions that adapt to changing conditions while preserving safety.**

The basic idea

During a surge (a flu outbreak, a trading crash, a supply-chain disruption), the system may need to grant someone a permission they don't normally have. But it must do so without opening a security hole.

The toolkit's **NeuralContribution** hook lets a neural component influence a decision:

```
from czoι import NeuralContribution

def surge_hook(user, operation, zone, base_decision):
    """Grant SeniorNurse the prescribe operation during a surge."""
    if (operation.name == "prescribe"
        and user.has_role("SeniorNurse")
        and zone.properties.get("surge_active", False)):
        return Decision.ALLOW
    return base_decision

builder.permission_engine.set_neural_contribution(
```

```
NeuralContribution(surge_hook)
)
```

The hook receives the base decision (the engine's normal result) and can return:

- The base decision — leave it unchanged.
- `Decision.ALLOW` — elevate.
- `Decision.DENY` — downgrade (lock down).

Toggleing adaptation at runtime

```
# Surge starts
builder.root.properties.set("surge_active", True)
builder.permission_engine.invalidate()

# Surge ends
builder.root.properties.set("surge_active", False)
builder.permission_engine.invalidate()
```

The cache invalidation ensures the change takes effect immediately.

Dynamic grants

An alternative to a hook: use the zone's `grant` method to permanently transfer a permission, then `revoke` it later.

```
# During surge
hospital.grant(senior_nurse_role, prescribe)

# After surge
hospital.revoke(senior_nurse_role, prescribe)
```

This is often preferable to a hook because the change is visible in the role's `base_permissions` — easy to audit.

Safety guarantees (Theorem 7)

An adaptive update is safe iff:

1. It preserves monotonicity — permissions only increase without explicit revocation.
2. It satisfies all identity and access constraints.
3. It maintains a complete audit trail.

The toolkit helps you satisfy these:

- `zone.grant` / `zone.revoke` are the only supported ways to change permissions at runtime.
- Every change is recorded when `audit_enabled=True`.

- Access constraints are evaluated on every decision.

When to use a hook vs a grant

Situation	Use
Temporary elevation during a well-defined event	Grant/revoke — clear boundaries, easy to audit.
Decision depends on complex state	Hook — flexible, no state changes needed.
Time-of-day or attribute-based policy	Hook — re-evaluated on every decision.
Cross-cutting policy applied to many roles	Constraint — expressed once in UniLang.

13. Audit and Observability

The audit trail

```
from czi import PermissionEngine

builder.permission_engine = PermissionEngine(
    cache_enabled=True,
    audit_enabled=True,
)
for zone in builder.root.walk():
    zone.set_permission_engine(builder.permission_engine)
```

Every call to `decide` records a `DecisionRecord`:

```
for r in builder.permission_engine.audit[-10:]:
    print(f"{r.timestamp:%H:%M:%S}  {r.user:<12} "
          f"{r.operation:<28} @{{r.zone:<16}} "
          f"→ {r.decision.name:<14} via {r.stage}")
```

Example output:

```
14:23:01  alice      EMR.prescribe      @Emergency      →
ALLOW      via local
14:23:02  bob        EMR.prescribe      @Emergency      →
INCONCLUSIVE via parent-override
14:23:03  bob        EMR.dispense       @Emergency      →
ALLOW      via local
```

Engine statistics

```
stats = builder.permission_engine.stats()
print(stats)
# {'hits': 43, 'misses': 1180, 'parent_lookups': 4, 'denies': 12}
```

- **hits** — decisions served from cache.
- **misses** — decisions that required engine evaluation.
- **parent_lookups** — number of times the recursive parent lookup fired.
- **denies** — total deny decisions.

Track these over time to detect performance regressions or unexpected denial spikes.

Daemon signals

Each daemon collects signals in its own fields:

```
class FleetDaemon(Daemon):
    def on_signal(self, signal, payload, source=None):
        if signal is DaemonSignal.STATE_WARNING:
            self.warnings.append((source.name, payload))
        elif signal is DaemonSignal.STATE_CRITICAL:
            self.criticals.append((source.name, payload))
```

Expose these as metrics for your observability stack:

```
# Prometheus-style
prometheus_client.Gauge("fleet_warnings_total").set(len(fleet.warnings))
prometheus_client.Gauge("fleet_criticals_total").set(len(fleet.criticals))
```

Integrating with structured logging

```
import logging
import json

logger = logging.getLogger("czoι")

def emit_audit(record):
    logger.info(json.dumps({
        "ts": record.timestamp.isoformat(),
        "user": record.user,
        "op": record.operation,
        "zone": record.zone,
        "decision": record.decision.name,
        "stage": record.stage,
    })))
```

```
for record in builder.permission_engine.audit:
    emit_audit(record)
```

14. Persistence and Web Frameworks

The toolkit is persistence-agnostic. Zones are runtime objects; persistence is a separate concern.

The pattern

1. **Runtime** — CZOI zones, roles, operations.
2. **Persistence** — ordinary ORM models that reference zones by name.
3. **Service** — glue code that reads persisted data, drives the runtime, writes results.

Example: Django

```
# models.py – plain Django models, nothing inheriting from Zone
from django.db import models

class Substation(models.Model):
    name = models.CharField(max_length=64, unique=True)
    voltage_kv = models.FloatField()
    capacity_mw = models.FloatField()

class SensorReading(models.Model):
    substation = models.ForeignKey(Substation, on_delete=models.CASCADE)
    timestamp = models.DateTimeField(auto_now_add=True, db_index=True)
    current_a = models.FloatField()
    voltage_kv = models.FloatField()
```

```
# runtime.py – build the CZOI runtime
from czoι import CZOABuilder

def build_runtime():
    builder = CZOABuilder("Grid")
    for sub_row in Substation.objects.all():
        zone = builder.add_zone(sub_row.name, parent=builder.root)
        zone.properties.set("voltage_kv", sub_row.voltage_kv)
        zone.properties.set("capacity_mw", sub_row.capacity_mw)
    return builder
```

```
# service.py – combine runtime + persistence
class GridService:
    def __init__(self):
        self.builder = build_runtime()
        self.engine = self.builder.permission_engine
```

```
def dispatch(self, user_name, operation_name):
    user = self.builder.root.users[user_name]
    op = self.builder.root.operations[operation_name]
    decision = self.engine.decide(user, op, self.builder.root)
    if decision.name == "ALLOW":
        SensorReading.objects.create(...)
    return decision
```

Example: FastAPI

```
from fastapi import FastAPI, Depends, HTTPException
from czoι import Decision

app = FastAPI()
service = GridService()

def current_user(token: str) -> str:
    # ... extract user name from token ...
    return "alice"

@app.post("/dispatch/{operation}")
def dispatch(operation: str, user: str = Depends(current_user)):
    decision = service.dispatch(user, operation)
    if decision is not Decision.ALLOW:
        raise HTTPException(403, detail=decision.name)
    return {"status": "ok"}
```

The toolkit imposes no framework requirements. The integration is a few lines of glue.

15. Complete Worked Example

Let's build a small hospital system end-to-end, exercising every feature.

The scenario

A regional health authority manages multiple hospitals. Each hospital has an Emergency department. During a flu outbreak, senior nurses need temporary authority to prescribe.

Step 1: Structure

```
from czoι import (
    Application, CZOABuilder, Daemon, DaemonSignal, Decision,
    Operation, Predictor, Role, User,
)

builder = CZOABuilder("RegionalHealthAuthority")
```

```

region = builder.root
hosp_a = builder.add_zone("CityHospitalA", parent=region)
emerg_a = builder.add_zone("Emergency", parent=hosp_a, atomic=True)
icu_a = builder.add_zone("ICU", parent=hosp_a, atomic=True)

```

Step 2: Operations and roles

```

app = Application("EMR", zone=hosp_a)
prescribe = app.add_operation(Operation("prescribe"))
dispense = app.add_operation(Operation("dispense"))
view = app.add_operation(Operation("view_patient"))
hosp_a.add_application(app)

attending = Role("AttendingPhysician", zone=hosp_a,
                 base_permissions=[prescribe, view])
senior_nurse = Role("SeniorNurse", zone=hosp_a,
                    base_permissions=[dispense, view])
nurse = Role("Nurse", zone=hosp_a,
              base_permissions=[dispense, view])
senior_nurse.add_junior(nurse)

hosp_a.add_role(attending)
hosp_a.add_role(senior_nurse)
hosp_a.add_role(nurse)

```

Step 3: Users

```

def register_along_path(user, zone):
    for z in zone.ancestry():
        if user.name not in z.users:
            z.add_user(user)

alice = User("alice", roles={"AttendingPhysician"})
bob = User("bob", roles={"SeniorNurse"})
carol = User("carol", roles={"Nurse"})

for u in [alice, bob, carol]:
    register_along_path(u, emerg_a)
    register_along_path(u, icu_a)

```

Step 4: Constraints

```

builder.add_access_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
    }

```

```

        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
    """)

```

This is a separation-of-duty rule: no single user should be able to both prescribe and dispense (to prevent fraud).

Step 5: Neural component

```

import numpy as np

sepsis_model = Predictor("sepsis", threshold=0.85)
X = np.array([
    [0.6, 0.4, 0.2], # normal vitals
    [0.9, 0.8, 0.9], # septic vitals
])
y = np.array([0.0, 1.0])
sepsis_model.fit(X, y, epochs=500)

hosp_a.add_neural("sepsis", sepsis_model)

```

Step 6: Daemons

```

class ClinicalSafetyDaemon(Daemon):
    def __init__(self, model, threshold=0.85, parent=None):
        super().__init__("ClinicalSafety", parent=parent, interval=1.0)
        self.model = model
        self.threshold = threshold

    def monitor(self):
        # In a real system, iterate over active patients
        # Here we simulate an alert
        score = self.model.predict({"hr": 0.9, "temp": 0.8, "lactate":
0.9})
        if score > self.threshold:
            self.emit_signal(
                DaemonSignal.STATE_WARNING,
                {"score": round(score, 3), "reason": "sepsis_risk"},
            )

class SurgeDaemon(Daemon):
    def __init__(self, hospital, parent=None):
        super().__init__("SurgeDaemon", parent=parent, interval=5.0)
        self.hospital = hospital
        self._active = False

    def monitor(self):
        arrivals = self.hospital.properties.get("recent_arrivals", 0)

```

```

        if not self._active and arrivals > 100:
            self._active = True
            self.emit_signal(
                DaemonSignal.STATE_CRITICAL,
                {"event": "surge_started", "arrivals": arrivals},
            )

class DirectorDaemon(Daemon):
    def __init__(self, hospital, ops, parent=None):
        super().__init__("Director", parent=parent, interval=1.0)
        self.hospital = hospital
        self.ops = ops
        self.surge_active = False

    def on_signal(self, signal, payload, source=None):
        if payload.get("event") == "surge_started" and not
self.surge_active:
            self.surge_active = True
            senior = self.hospital.roles["SeniorNurse"]
            self.hospital.grant(senior, self.ops["prescribe"])

```

Step 7: Wire it up

```

director = DirectorDaemon(hosp_a, {"prescribe": prescribe})
safety = ClinicalSafetyDaemon(sepsis_model, parent=director)
surge = SurgeDaemon(hosp_a, parent=director)

builder.add_daemon(director)
builder.add_daemon(safety)
builder.add_daemon(surge)

```

Step 8: Run it

```

engine = builder.permission_engine

# Normal conditions
print("Normal:")
print(f"  bob prescribe: {engine.decide(bob, prescribe, hosp_a).name}")
# INCONCLUSIVE – SeniorNurse doesn't have prescribe

# Simulate a surge
hosp_a.properties.set("recent_arrivals", 200)
for _ in range(10):
    builder.daemon_manager.tick()

print("\nDuring surge:")
print(f"  bob prescribe: {engine.decide(bob, prescribe, hosp_a).name}")
# ALLOW – SurgeDaemon triggered DirectorDaemon to grant prescribe

```

```
# Audit trail
print("\nAudit:")
for r in builder.permission_engine.audit:
    print(f" {r.user} {r.operation} @{r.zone} → {r.decision.name}")
```

What this example demonstrates

- **Recursive zones** — region → hospital → emergency/ICU.
- **Seniority inheritance** — SeniorNurse inherits Nurse's permissions.
- **Separation of duty** — no user can both prescribe and dispense.
- **Neural component** — sepsis predictor trained on synthetic vitals.
- **Hierarchical daemons** — SurgeDaemon signals DirectorDaemon, which dynamically grants a permission.
- **Adaptive access control** — Bob's effective permissions change in real time.
- **Audit trail** — every decision is recorded.

16. Cookbook

Find all zones where a user has a role

```
def zones_with_role(user, role_name):
    return [z for z in builder.root.walk()
            if role_name in user.roles and role_name in z.roles]

# Example
print(zones_with_role(bob, "SeniorNurse"))
```

List every operation a user can perform

```
def accessible_operations(user, zone):
    engine = builder.permission_engine
    return [op.qualified_name for op in zone.operations.values()
            if engine.decide(user, op, zone) is Decision.ALLOW]

print(accessible_operations(alice, emergency))
```

Find the least-senior role that can perform an operation

```
def least_role(zone, operation):
    candidates = [r for r in zone.roles.values()
                  if operation in r.base_permissions]
    if not candidates:
        return None
    # The role with the fewest juniors is the "least senior"
```

```

        return min(candidates, key=lambda r: len(r.junior_roles))

    print(least_role(hosp_a, prescribe).name)    # "AttendingPhysician"

```

Build a role-hierarchy report

```

def role_tree(zone, role, indent=0):
    print(" " * indent + role.name)
    for junior in sorted(role.junior_roles, key=lambda r: r.name):
        role_tree(zone, junior, indent + 1)

# Find top-level roles (those with no seniors)
def top_roles(zone):
    all_juniors = {r for role in zone.roles.values()
                  for r in role.junior_roles}
    return [r for r in zone.roles.values() if r not in all_juniors]

for role in top_roles(hosp_a):
    role_tree(hosp_a, role)

```

Snapshot the system to JSON

```

import json

def snapshot(builder):
    def zone_repr(zone):
        return {
            "name": zone.name,
            "roles": list(zone.roles.keys()),
            "operations": list(zone.operations.keys()),
            "users": list(zone.users.keys()),
            "properties": zone.properties.as_dict(),
            "children": [zone_repr(c) for c in zone.zones.values()],
        }
    return zone_repr(builder.root)

print(json.dumps(snapshot(builder), indent=2))

```

Trace a permission decision step by step

```

def trace_decision(engine, user, operation, zone):
    print(f"Trace: {user.name} → {operation.qualified_name} @ {zone.name}")
    for z in [zone] + zone.ancestry()[::-1]:
        decision = engine.evaluate_local(user, operation, z)
        print(f"  @{z.name}: {decision.name}")

```

```
if decision is not Decision.INCONCLUSIVE:
    break
```

Block all operations for a user (security lockdown)

```
def lockdown(user, zone):
    for role_name in list(user.roles):
        role = zone.roles.get(role_name)
        if role is None:
            continue
        for op in list(role.base_permissions):
            zone.revoke(role, op)
    user.attributes.set("locked_down", True)

lockdown(bob, hosp_a)
```

Undo a lockdown

```
def unlock(user, zone, saved_permissions):
    for role_name, ops in saved_permissions.items():
        role = zone.roles.get(role_name)
        if role is None:
            continue
        for op in ops:
            zone.grant(role, op)
    user.attributes.set("locked_down", False)
```

Round-trip a zone tree from a config file

```
# zones.yaml
# zones:
#   - name: Hospital
#     children:
#       - name: Emergency
#       - name: ICU
#       - name: Surgery

import yaml

def from_yaml(builder, path):
    data = yaml.safe_load(open(path))
    def build(specs, parent):
        for spec in specs:
            zone = builder.add_zone(spec["name"], parent=parent)
            if spec.get("children"):
                build(spec["children"], zone)

    build(data["zones"], builder)
```

```
        build(spec["children"], parent=zone)
    build(data["zones"], builder.root)
```

17. Troubleshooting

ZoneContainmentError: User X must be affiliated with parent Y

You tried to register a user in a child zone without registering them in the parent first.

Fix:

```
# Wrong
emergency.add_user(alice)

# Right
for z in emergency.ancestry():
    z.add_user(alice)
```

TypeError: AtomicZone cannot have child zones

You tried to add a child to a leaf. Make the zone composite (the default) or don't add children to it.

```
# Wrong
leaf = builder.add_zone("Leaf", parent=parent, atomic=True)
leaf.add_zone(builder.add_zone("Child")) # raises

# Right
leaf = builder.add_zone("Leaf", parent=parent) # composite
leaf.add_zone(builder.add_zone("Child"))
```

UniLogBridgeError: The unilog-toolkit is required

Install `unilog-toolkit`:

```
pip install unilog-toolkit
```

UniLangSyntaxError: Unexpected character X

Check for:

- Missing semicolons after signature declarations.
- Comments using `//` instead of `#` or `%`.
- Unicode operators (`^` instead of `&` or `and`).

UniLang accepts both ASCII and Unicode, but mixing them mid-expression can confuse the parser.

Decision.INCONCLUSIVE when you expect DENY

INCONCLUSIVE means "no covering role." DENY means "covering role, but a constraint blocked it." These are semantically different — the toolkit distinguishes them on purpose.

If you want boolean semantics, compare explicitly:

```
allowed = engine.decide(user, op, zone) is Decision.ALLOW
```

Cache staleness

If a permission change doesn't seem to take effect:

```
# Ensure you used the zone-level method
hospital.grant(role, operation) # invalidates cache
# Not:
role.grant(operation)           # does NOT invalidate cache
```

Or invalidate manually:

```
builder.permission_engine.invalidate()
```

Daemon not firing

Check:

1. You called `builder.add_daemon(daemon)`.
2. `daemon.enabled` is `True`.
3. `daemon.interval` is set appropriately.
4. You're calling `daemon_manager.tick()` (sync) or `daemon_manager.run()` (async).

```
print(daemon.enabled, daemon.interval)
print([d.name for d in builder.daemon_manager.daemons])
```

Neural model predictions are nonsense

Most commonly:

1. Features aren't normalised (z-score everything before fitting).
2. Training data has no signal (verify with a simple baseline first).
3. Threshold is miscalibrated (start with the auto-calibrated value).

```
print(f"Mean positive score: {X_pos.mean(axis=0)}")
print(f"Mean negative score: {X_neg.mean(axis=0)}")
# The two should differ substantially.
```

18. Best Practices

Structure

- **Model the organisation, not the code.** If the HR department is a zone, so be it — even if it only has two users.
- **Use atomic zones generously.** Any zone without children should be `atomic=True`. This catches accidental additions early.
- **Name operations with verb_noun.** `view_patient`, `process_payroll`, `disconnect_feeder`. Not `viewPatient` or `VP`.

Roles

- **One role per job function.** If you find yourself writing `role_that_can_do_X_and_Y`, you probably want a seniority relation instead.
- **Prefer seniority over duplication.** If Doctor and SeniorDoctor share 90 % of their permissions, model the shared permissions on Doctor and give SeniorDoctor its extra permissions.
- **Keep role hierarchies shallow.** Depth 3 is usually enough. Deeper hierarchies are hard to reason about.

Operations

- **Atomic operations, not composite actions.** `submit_grade` is atomic; `finalise_semester` is not (it's a workflow).
- **Fine granularity for security-critical operations.** If you audit `edit_record` you can't tell whether the edit was a typo fix or a fraud. Prefer `edit_salary`, `edit_address`, `edit_bank_account`.
- **Application grouping is for humans, not for permissions.** Applications exist to organise the UI and deployment; permissions are always on operations.

Constraints

- **Encode policy, not implementation.** "A user cannot both prescribe and dispense" is policy. "If `user.shift == 'day'` and `time.hour > 8`" is implementation. Prefer the former.
- **One constraint per rule.** Don't combine unrelated constraints into a single UniLang formula.
- **Test constraints in isolation.** Write a unit test for each constraint that verifies it blocks what it should and allows what it should.

Daemons

- **One responsibility per daemon.** `BatteryDaemon` monitors battery. `OvercurrentDaemon` monitors current. Don't combine.

- **Signal, don't act.** A daemon's `monitor()` should emit signals; the parent daemon's `on_signal()` should act. This keeps the tree composable.
- **Use per-daemon intervals.** A battery check might need 500 ms; a compliance audit might need 60 s. Don't force a single interval.

Neural components

- **Normalise features.** Always. The autoencoder especially.
- **Fit on normal data only.** Anomaly detection works by learning the normal manifold; including anomalies in training destroys the signal.
- **Version your models.** Record which model produced which decision in the audit trail. This is essential for post-incident review.

Testing

- **Test with a fresh builder per test.** Don't share state across tests.
- **Test `Decision.ALLOW`, `Decision.DENY`, and `Decision.INCONCLUSIVE` separately.** They mean different things.
- **Test constraints explicitly.** A test that only exercises `ALLOW` doesn't prove the `DENY` path works.

Performance

- **Cache is your friend.** For production workloads, keep `cache_enabled=True` and monitor `stats()["hits"]`.
- **Avoid wide fan-outs.** If a role has 10 000 base permissions, every decision walks them. Prefer smaller roles with seniority relations.
- **Deep trees are fine.** Recursion depth adds ~0.02 ms per level. Trees of depth 10+ are common and fast.

19. API Reference

CZOABuilder

Method	Description
<code>CZOABuilder(name)</code>	Create a builder with a root composite zone
<code>.root</code>	The root <code>CompositeZone</code>
<code>.add_zone(name, parent=None, atomic=False)</code>	Add a zone; parent defaults to root
<code>.permission_engine</code>	The shared <code>PermissionEngine</code>
<code>.constraint_manager</code>	The shared <code>ConstraintManager</code>
<code>.daemon_manager</code>	The shared <code>DaemonManager</code>
<code>.embedding_service</code>	The shared <code>EmbeddingService</code>
<code>.add_access_constraint(unilang)</code>	Register an access constraint

Method	Description
<code>.add_identity_constraint(unilang)</code>	Register an identity constraint
<code>.add_trigger_constraint(unilang)</code>	Register a trigger constraint
<code>.add_goal_constraint(unilang)</code>	Register a goal constraint
<code>.register_predicate(name, fn)</code>	Register a custom predicate for constraints
<code>.add_daemon(daemon)</code>	Register a daemon with the root and manager

ZoneBase (base for `AtomicZone`, `CompositeZone`)

Attribute / Method	Description
<code>.name</code>	Zone name
<code>.parent</code>	Parent zone (None for root)
<code>.zones</code>	<code>dict[str, ZoneBase]</code> of children
<code>.roles</code>	<code>dict[str, Role]</code>
<code>.users</code>	<code>dict[str, User]</code>
<code>.applications</code>	<code>dict[str, Application]</code>
<code>.operations</code>	<code>dict[str, Operation]</code>
<code>.neural</code>	<code>dict[str, Any]</code>
<code>.embeddings</code>	The embedding service
<code>.constraints</code>	The constraint manager
<code>.permission_engine</code>	The permission engine
<code>.daemons</code>	<code>list[Daemon]</code>
<code>.properties</code>	<code>PropertyStore</code>
<code>.add_zone(child)</code>	Register a child (raises on atomic)
<code>.add_role(role)</code>	Register a role; propagates to children
<code>.add_user(user)</code>	Register a user (enforces containment)
<code>.add_application(app)</code>	Register an application and its operations
<code>.add_operation(op)</code>	Register a bare operation
<code>.add_neural(name, component)</code>	Register a neural component
<code>.add_daemon(daemon)</code>	Register a daemon with this zone
<code>.grant(role, operation)</code>	Grant and invalidate cache

Attribute / Method	Description
<code>.revoke(role, operation)</code>	Revoke and invalidate cache
<code>.walk()</code>	Iterate this zone and all descendants
<code>.ancestry()</code>	List from root down to this zone
<code>.depth()</code>	Depth from the root (root is 0)
<code>.is_ancestor_of(other)</code>	True if <code>self</code> is an ancestor of <code>other</code>

Role

Method	Description
<code>Role(name, zone=None, base_permissions=[], attributes={})</code>	Construct
<code>.name, .zone</code>	Identity
<code>.base_permissions</code>	<code>set[Operation]</code>
<code>.junior_roles</code>	<code>set[Role]</code>
<code>.grant(op) / .revoke(op)</code>	Modify base permissions directly (no cache invalidation)
<code>.add_junior(role)</code>	Declare this role senior to <code>role</code>
<code>.qualified_name</code>	<code>"<zone>:<name>"</code>

User

Method	Description
<code>User(name, roles=set(), attributes={}, credentials={})</code>	Construct
<code>.name, .roles, .attributes, .credentials</code>	State
<code>.has_role(name)</code>	Membership test

Operation, Application

Method	Description
<code>Operation(name, application=None, properties={})</code>	Construct
<code>.qualified_name</code>	<code>"<app>.<name>"</code>
<code>Application(name, zone=None, properties={})</code>	Construct
<code>.add_operation(op)</code>	Register an operation (sets <code>.application</code>)

PermissionEngine

Method	Description
<code>PermissionEngine(cache_enabled=True, audit_enabled=False)</code>	Construct
<code>.decide(user, operation, zone)</code>	Returns Decision
<code>.evaluate_local(user, operation, zone)</code>	Single-stage decision (no recursion)
<code>.invalidate()</code>	Clear the decision cache
<code>.stats()</code>	Dict with hits , misses , parent_lookups , denies
<code>.audit</code>	List of DecisionRecord (if enabled)
<code>.set_neural_contribution(contribution)</code>	Attach a NeuralContribution hook

Decision

Enum: **Decision.ALLOW**, **Decision.DENY**, **Decision.INCONCLUSIVE**.

ConstraintManager

Method	Description
<code>.add(kind, source, name=None)</code>	Parse and register a UniLang constraint
<code>.add_access(source) / .add_identity(...) / .add_trigger(...) / .add_goal(...)</code>	Convenience methods
<code>.register_predicate(name, fn)</code>	Custom predicate
<code>.check_all(request=None)</code>	Evaluate all constraints; return dict[kind, bool]
<code>.check_zone(zone=None, request=None)</code>	Same, scoped to a zone
<code>.is_satisfied(user, operation, zone)</code>	Hook used by the permission engine
<code>.attach(zone)</code>	Bind to the root zone (called automatically)

Daemon

Method	Description
<code>Daemon(name, parent=None, interval=1.0)</code>	Construct
<code>.monitor()</code>	Override — periodic work

Method	Description
<code>.on_signal(signal, payload, source=None)</code>	Override — handle child signals
<code>.emit_signal(signal, payload)</code>	Emit upward
<code>.start() / .stop()</code>	Enable / disable
<code>.safe_monitor()</code>	Error-isolated monitor (called by manager)
<code>.register_with(zone)</code>	Called automatically

DaemonSignal

Enum: `STATE_NORMAL`, `STATE_WARNING`, `STATE_CRITICAL`, `REVOKE`, `ESCALATE`.

DaemonManager

Method	Description
<code>DaemonManager(default_interval=1.0, max_workers=4)</code>	Construct
<code>.add(daemon) / .extend(daemons)</code>	Register
<code>.tick()</code>	Synchronous single-pass
<code>await .run(duration=None)</code>	Async loop
<code>.stop()</code>	Stop the loop
<code>.shutdown()</code>	Stop and shut down the executor

Predictor

Method	Description
<code>Predictor(name, fn=None, threshold=0.5, input_dim=None)</code>	Construct
<code>.fit(X, y, epochs=200, lr=0.05)</code>	Train
<code>.predict(features)</code>	<code>float</code> in <code>[0, 1]</code>
<code>.fires(features)</code>	<code>bool</code> (<code>predict >= threshold</code>)

AnomalyDetector

Method	Description
<code>AnomalyDetector(name, input_dim, latent_dim=8, threshold=0.1, seed=0)</code>	Construct
<code>.fit(X, epochs=200, lr=0.05)</code>	Train and calibrate threshold

Method	Description
<code>.score(x)</code>	Reconstruction error (Float)
<code>.is_anomalous(x)</code>	<code>bool</code>

RoleMiner

Method	Description
<code>RoleMiner(latent_dim=16, min_cluster_size=3, seed=0, ...)</code>	Construct
<code>.mine(X, operation_names, min_support=0.5)</code>	Returns <code>MiningResult</code>

`MiningResult` fields: `suggested_roles`, `confidence`, `n_clusters`.

EmbeddingService

Method	Description
<code>EmbeddingService(dimension=64, model_name=..., use_transformer=False)</code>	Construct
<code>.embed(text)</code>	Vector for a string
<code>.embed_operation(op) / .embed_role(role)</code>	Entity-specific embeddings
<code>.align_to_global(v)</code>	Project a local vector into the shared space
<code>.train_alignment(positives, negatives, epochs, lr, margin)</code>	Fit the alignment layer
<code>.similarity(a, b)</code>	Cosine similarity
<code>.save_alignment(path) / .load_alignment(path)</code>	Persistence

NeuralContribution

```
NeuralContribution(fn: Callable[[user, operation, zone, base], Decision])
```

Attach with `builder.permission_engine.set_neural_contribution(...)`.

Exceptions

Exception	Raised when
<code>CZOIError</code>	Base class
<code>ZoneError</code>	Invalid zone operation

Exception	Raised when
<code>ZoneContainmentError</code>	$U_child \subseteq U_parent$ violated
<code>RoleError</code>	Role definition problem
<code>UserError</code>	User definition problem
<code>ApplicationError</code>	Application problem
<code>PermissionError</code>	Permission calculus violation
<code>ConstraintError</code>	Constraint manager problem
<code>SafetyViolation</code>	Adaptive update violated a safety invariant
<code>UniLogBridgeError</code>	UniLog toolkit unavailable
<code>UniLangSyntaxError</code>	Parse error in UniLang source
<code>UniLangSemanticError</code>	Undefined sort, arity mismatch, etc.
<code>UniLangEvaluationError</code>	Runtime evaluation error

Getting Help

- **Examples:** the `examples/` directory in the repository.
- **Tests:** `tests/test_basic.py` shows every core feature in use.
- **Issues:** github.com/hongxueharriswang/czoi-toolkit/issues
- **UniLog:** github.com/hongxueharriswang/unilog-toolkit

End of User Guide.